

BUSINESS REQUIREMENTS DOCUMENT

Insurance Policy Management System

Intern Software Development Project

Document Version	1.0
Prepared For	Intern Developer
Domain	Insurance — Policy Management
Tech Stack	Java Spring Boot PostgreSQL Angular
Complexity	Intermediate (APIs + Workflows + Auth)
Date	June 2026

1. Project Overview

1.1 Purpose

This document defines the business and technical requirements for building an Insurance Policy Management System (IPMS). It serves as the primary reference for the intern developer throughout the development lifecycle.

1.2 Project Background

Insurance companies manage thousands of policies covering customers across life, health, motor, and property categories. Manual tracking of policy lifecycle — from issuance to renewal and claims linkage — is error-prone and inefficient. IPMS aims to digitize and streamline this workflow for a mid-sized insurance firm.

1.3 Objectives

- Build a full-stack web application for policy lifecycle management
- Implement secure role-based access for Admin, Agent, and Customer roles
- Provide RESTful APIs with proper authentication and authorization
- Enable policy creation, renewal, suspension, and cancellation workflows
- Deliver a responsive Angular frontend for all user interactions
- Persist all data in PostgreSQL with a well-designed relational schema

1.4 Scope

In Scope:

- Policy CRUD operations
- Customer and Agent management
- Policy lifecycle state machine (Draft → Active → Renewed / Expired / Cancelled)
- JWT-based authentication and role-based authorization
- Basic premium calculation engine
- Policy document upload and storage
- Notification (in-app) on policy state changes

Out of Scope:

- Claims processing module (future phase)
- Payment gateway integration
- Mobile app

- Third-party insurer API integrations

1.5 Stakeholders

Stakeholder	Role	Responsibility
Mentor / Tech Lead	Reviewer	Code review, architecture guidance, sprint sign-off
Intern Developer	Developer	Design, develop, test, and deploy the system
Business Analyst	Requirements	Clarify domain requirements and acceptance criteria
QA Engineer	Testing	Validate functional and non-functional requirements

2. Functional Requirements

2.1 User Roles & Access Control

Role	Users	Permissions
ADMIN	System administrators	Full access: manage all users, policies, agents, reports
AGENT	Insurance agents/brokers	Create & manage policies for their customers, view own portfolio
CUSTOMER	Policyholders	View own policies, download documents, request renewals

2.2 Authentication & Authorization

- FR-AUTH-01: Users can register with email, password, and role assignment by admin
- FR-AUTH-02: Users log in with email and password, receiving a JWT token (expiry: 24h)
- FR-AUTH-03: Refresh token mechanism to extend sessions without re-login
- FR-AUTH-04: All protected API endpoints require a valid Bearer token
- FR-AUTH-05: Role-based access enforced on both backend (Spring Security) and frontend (Angular route guards)
- FR-AUTH-06: Passwords stored as bcrypt hashes; minimum 8 characters with complexity rules
- FR-AUTH-07: Account lockout after 5 consecutive failed login attempts

2.3 Policy Management

2.3.1 Policy Types

- Life Insurance
- Health Insurance
- Motor Insurance
- Property Insurance

2.3.2 Policy Lifecycle States

A policy follows a strict state machine:

From State	To State	Trigger / Condition
DRAFT	ACTIVE	Agent/Admin activates after premium payment confirmation
ACTIVE	RENEWED	Renewal request submitted and processed before expiry date
ACTIVE	EXPIRED	Expiry date passed without renewal
ACTIVE	SUSPENDED	Non-payment or compliance flag raised by admin

SUSPENDED	ACTIVE	Issue resolved and reinstated by admin
ANY	CANCELLED	Customer or Agent submits cancellation request, Admin approves

2.3.3 Policy Operations

- FR-POL-01: Agent/Admin can create a new policy in DRAFT state
- FR-POL-02: System auto-generates a unique policy number (format: IPMS-YYYY-XXXXXXX)
- FR-POL-03: Premium auto-calculated based on type, sum insured, age, and duration
- FR-POL-04: Agent/Admin can upload policy documents (PDF, max 5 MB)
- FR-POL-05: Customer can view and download their own policy documents
- FR-POL-06: Admin can search/filter policies by status, type, agent, customer, date range
- FR-POL-07: Policies expiring within 30 days shown in the renewal dashboard
- FR-POL-08: Full audit trail maintained for every state change with timestamp and actor

2.4 Customer Management

- FR-CUST-01: Admin/Agent can create and update customer profiles
- FR-CUST-02: Customer profile includes: name, DOB, gender, contact info, KYC documents
- FR-CUST-03: Each customer has a unique customer ID (CUST-XXXXXXX)
- FR-CUST-04: Admin can deactivate/reactivate customer accounts
- FR-CUST-05: Customer can view all their own policies from the dashboard

2.5 Agent Management

- FR-AGT-01: Admin can create and manage agent accounts
- FR-AGT-02: Each agent has a unique agent code and their own portfolio view
- FR-AGT-03: Admin can view per-agent policy count, active policies, and premium volume
- FR-AGT-04: An agent can be assigned/unassigned from policies

2.6 Notifications

- FR-NOTIF-01: In-app notification generated on every policy state transition
- FR-NOTIF-02: Notification bell with unread count visible in navbar
- FR-NOTIF-03: Users can mark notifications as read; older ones auto-archived after 90 days

3. Technical Architecture

3.1 Technology Stack

Layer	Technology	Purpose
Frontend	Angular 17+ (TypeScript)	SPA, reactive UI, route guards
UI Library	Angular Material / PrimeNG	Pre-built components, theming
Backend	Java 21, Spring Boot 3.x	REST API, business logic, security
Security	Spring Security + JWT (JJWT)	Auth, roles, token management
ORM	Spring Data JPA (Hibernate)	Database access layer
Database	PostgreSQL 15+	Relational data persistence
Migrations	Flyway	Version-controlled DB schema changes
Build Tool	Maven (Backend), npm / Angular CLI (Frontend)	Build, dependency management
Testing	JUnit 5, Mockito, Jasmine, Karma	Unit and integration tests
API Docs	Springdoc OpenAPI (Swagger UI)	Auto-generated API documentation
File Storage	Local filesystem / MinIO (S3 compatible)	Policy document storage
Version Control	Git + GitHub	Source control, PR-based workflow

3.2 System Architecture

The system follows a layered monolithic architecture suitable for an intern-level project, with clear separation of concerns:

Frontend (Angular SPA):

- Components → Services → HTTP Client → Backend APIs
- AuthGuard & RoleGuard on routes
- Interceptor for attaching JWT to all outgoing requests

Backend (Spring Boot):

- Controller Layer — REST endpoints, request/response mapping
- Service Layer — Business logic, state machine, validation
- Repository Layer — Spring Data JPA repositories
- Security Layer — JWT filter chain, method-level security
- DTO Layer — Request/Response DTOs with Bean Validation

Database (PostgreSQL):

- Normalized relational schema (3NF)
- Flyway migration scripts for all DDL changes
- Indexes on foreign keys, policy_number, status columns

3.3 Security Architecture

- HTTPS enforced in production; HTTP in local dev only
- JWT tokens signed with HS256 (secret key from environment variable)
- Tokens carry: sub (userId), roles, iat, exp claims
- Spring Security @PreAuthorize annotations on service methods
- CORS configured to allow only the Angular dev/prod origin
- Input validation via Jakarta Bean Validation (javax annotations)
- SQL injection prevented by JPA parameterized queries

4. Database Schema

All tables include created_at, updated_at (timestamp with time zone) and created_by columns for auditing unless otherwise noted.

4.1 Table: users

Column	Type	Constraints	Description
id	BIGSERIAL	PK	Auto-increment primary key
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hashed password
role	VARCHAR(20)	NOT NULL	ADMIN / AGENT / CUSTOMER
is_active	BOOLEAN	DEFAULT TRUE	Account active flag
failed_attempts	INTEGER	DEFAULT 0	Lockout tracking
last_login	TIMESTAMPTZ	NULLABLE	Last successful login

4.2 Table: customers

Column	Type	Constraints	Description
id	BIGSERIAL	PK	Primary key
customer_code	VARCHAR(20)	UNIQUE, NOT NULL	CUST-XXXXXXX format
user_id	BIGINT	FK → users.id	Linked user account
full_name	VARCHAR(200)	NOT NULL	Full legal name
date_of_birth	DATE	NOT NULL	For age calculation
gender	VARCHAR(10)	NOT NULL	MALE/FEMALE/OTHER
phone	VARCHAR(20)	NOT NULL	Contact number
address	TEXT	NULLABLE	Residential address
kyc_status	VARCHAR(20)	DEFAULT 'PENDING'	PENDING/VERIFIED/REJECTED

4.3 Table: agents

Column	Type	Constraints	Description
--------	------	-------------	-------------

id	BIGSERIAL	PK	Primary key
agent_code	VARCHAR(20)	UNIQUE, NOT NULL	AGT-XXXXXXX format
user_id	BIGINT	FK → users.id	Linked user account
full_name	VARCHAR(200)	NOT NULL	Agent full name
license_number	VARCHAR(50)	UNIQUE, NOT NULL	Regulatory license
is_active	BOOLEAN	DEFAULT TRUE	Active flag

4.4 Table: policies

Column	Type	Constraint s	Description
id	BIGSERIAL	PK	Primary key
policy_number	VARCHAR(30)	UNIQUE, NOT NULL	IPMS-YYYY-XXXXXXX
policy_type	VARCHAR(30)	NOT NULL	LIFE/HEALTH/MOTOR/PROPERTY
status	VARCHAR(20)	NOT NULL	DRAFT/ACTIVE/RENEWED/EXPIRED/SUSPENDED/CANCELLED
customer_id	BIGINT	FK → customers. id	Policyholder
agent_id	BIGINT	FK → agents.id	Assigned agent
sum_insured	NUMERIC(15, 2)	NOT NULL	Coverage amount
premium_amount	NUMERIC(10, 2)	NOT NULL	Annual premium (calculated)
start_date	DATE	NOT NULL	Coverage start
end_date	DATE	NOT NULL	Coverage end/expiry
description	TEXT	NULLABLE	Coverage notes

4.5 Table: policy_audit_log

Column	Type	Constraints	Description
id	BIGSERIAL	PK	Primary key
policy_id	BIGINT	FK →	Related policy

		policies.id	
previous_status	VARCHAR(20)	NULLABLE	State before change
new_status	VARCHAR(20)	NOT NULL	State after change
changed_by	BIGINT	FK → users.id	Actor user
remarks	TEXT	NULLABLE	Reason for change
changed_at	TIMESTAMPTZ	NOT NULL	Timestamp of transition

4.6 Additional Tables (Brief)

Table Name	Key Columns & Purpose
policy_documents	id, policy_id (FK), file_name, file_path, uploaded_by, uploaded_at — stores metadata for uploaded PDF files
notifications	id, user_id (FK), title, message, is_read, created_at — in-app notification store
refresh_tokens	id, user_id (FK), token (hashed), expiry, is_revoked — refresh token management

5. API Endpoints

Base URL: /api/v1 | All endpoints return JSON | Auth header: Authorization: Bearer <token>

5.1 Authentication APIs

Method	Endpoint	Auth	Description
POST	/auth/register	Public	Register new user (Admin use only in prod)
POST	/auth/login	Public	Returns JWT + refresh token
POST	/auth/refresh	Public	Exchange refresh token for new JWT
POST	/auth/logout	Bearer	Revoke refresh token

5.2 Customer APIs

Method	Endpoint	Roles	Description
GET	/customers	ADMIN, AGENT	List all customers (paginated)
GET	/customers/{id}	ADMIN, AGENT, CUST*	Get customer by ID (* own profile only)
POST	/customers	ADMIN, AGENT	Create new customer profile
PUT	/customers/{id}	ADMIN, AGENT	Update customer details
DELETE	/customers/{id}	ADMIN	Soft-delete (deactivate) customer

5.3 Policy APIs

Method	Endpoint	Roles	Description
GET	/policies	ADMIN, AGENT	List all policies (filterable)
GET	/policies/{id}	ALL (ownership check)	Get policy details
POST	/policies	ADMIN, AGENT	Create new policy (DRAFT state)
PUT	/policies/{id}	ADMIN, AGENT	Update policy details (DRAFT only)
PATCH	/policies/{id}/activate	ADMIN,	Transition DRAFT → ACTIVE

		AGENT	
PATCH	/policies/{id}/renew	ADMIN, AGENT	Transition ACTIVE → RENEWED
PATCH	/policies/{id}/suspend	ADMIN	Transition ACTIVE → SUSPENDED
PATCH	/policies/{id}/cancel	ADMIN, AGENT	Transition ANY → CANCELLED
GET	/policies/{id}/audit	ADMIN, AGENT	Get full audit trail for a policy
GET	/customers/{id}/policies	ALL (ownership check)	List all policies for a customer
GET	/policies/expiring-soon	ADMIN, AGENT	Policies expiring in 30 days

5.4 Document & Notification APIs

Method	Endpoint	Roles	Description
POST	/policies/{id}/documents	ADMIN, AGENT	Upload policy document (multipart)
GET	/policies/{id}/documents	ALL (ownership)	List documents for a policy
GET	/documents/{docId}/download	ALL (ownership)	Download a document file
GET	/notifications	ALL (own only)	Get my notifications (paginated)
PATCH	/notifications/{id}/read	ALL (own only)	Mark notification as read

6. UI / UX Wireframe Descriptions

All screens should be implemented as Angular components using a responsive grid layout. Use Angular Material or PrimeNG for component library.

6.1 Login Page

- Centered card layout (no sidebar)
- Fields: Email, Password; 'Show password' toggle
- Primary CTA: 'Sign In' button
- Error messages inline below fields for invalid credentials or locked accounts
- 'Forgot password' placeholder link (not implemented in intern scope)

6.2 Dashboard (Role-specific)

Admin Dashboard:

- Top KPI cards: Total Policies, Active Policies, Expiring Soon (30d), Total Customers
- Recent Activity table: last 10 policy state changes
- Quick links: Create Policy, Add Customer, Add Agent

Agent Dashboard:

- KPI cards: My Policies, My Active Policies, Renewal Due
- My Policy list with status badges

Customer Dashboard:

- My Policies cards with type icon, status badge, expiry date
- Quick action: 'View Documents' per policy

6.3 Policy List Page

- Top filter bar: Policy Type (dropdown), Status (dropdown), Date Range (date picker), Search by Policy Number or Customer Name
- Data table columns: Policy No., Type, Customer, Agent, Premium, Start Date, End Date, Status, Actions
- Status shown as colored chips (green = ACTIVE, orange = DRAFT, red = CANCELLED/EXPIRED, blue = RENEWED)
- Actions column: View, Edit (if DRAFT), and status-transition buttons (Activate / Renew / Suspend / Cancel) based on role and current state
- Pagination: 10/25/50 rows per page

6.4 Policy Detail Page

- Two-column layout: left for policy info, right for documents and audit trail
- Policy Info section: all fields in read-only card; 'Edit' button (if DRAFT and authorized)
- Status transition action bar: contextual buttons based on current state and role
- Documents section: list of uploaded files with download icon; 'Upload Document' button for agents
- Audit Trail section: timeline view with icon per state, actor name, timestamp, remarks

6.5 Create / Edit Policy Form

- Step 1 — Basic Details: Policy Type (radio cards), Customer (autocomplete search), Agent (autocomplete)
- Step 2 — Coverage Details: Sum Insured, Start Date, End Date, Description
- Step 3 — Preview & Submit: read-only summary with calculated premium; 'Save as Draft' and 'Activate' buttons
- Real-time premium preview updates as user changes sum insured or date range

6.6 Customer & Agent Management

- Tabular list with search/filter by name, status, KYC status
- 'Add New' button opens a slide-out panel form (not modal) for creation
- Row actions: View, Edit, Deactivate/Reactivate

6.7 Common Navigation

- Sidebar navigation (collapsible on mobile): Dashboard, Policies, Customers, Agents (Admin/Agent only), Notifications, Logout
- Top header: App logo, user avatar + name, notification bell with badge count
- Breadcrumb trail on all inner pages

7. Non-Functional Requirements

Category	Requirement	Details
Performance	API Response Time	95th percentile < 500ms for list queries; < 200ms for single-record fetches
Scalability	Pagination	All list APIs must support pagination (page, size params); no unbounded queries
Security	OWASP Top 10	Address SQL injection, XSS, CSRF, broken auth; use Spring Security defaults
Reliability	Error Handling	Global exception handler; structured JSON error responses with status code, message, timestamp
Usability	Responsive UI	Functional on desktop (1280px+) and tablet (768px+); Angular Material breakpoints
Maintainability	Code Quality	Checkstyle for Java; ESLint for Angular; minimum 60% unit test coverage
Documentation	API Docs	Swagger UI available at /swagger-ui.html; all endpoints annotated
Logging	Structured Logs	SLF4J + Logback; log all state transitions and security events at INFO/WARN level

8. Timeline & Milestones

Estimated internship duration: 10 weeks. Each sprint is 1 week. Deliverables are reviewed at end of each sprint with the mentor.

Sprint	Week	Deliverables	Acceptance Criteria
1	Week 1	Project setup, DB schema, Flyway migrations, Spring Boot skeleton with health endpoint	DB tables created; /actuator/health returns UP; Git repo initialized
2	Week 2	Auth module: register, login, JWT generation, refresh token, Spring Security config	Postman tests pass for all auth endpoints; unauthorized calls return 401
3	Week 3	Customer and Agent CRUD APIs; role-based access enforcement	All CRUD endpoints work; role violations return 403; unit tests written
4	Week 4	Policy creation, policy number generation, premium calculator, DRAFT state API	Policy created with auto-generated number; premium calculated correctly
5	Week 5	Policy state machine (activate, renew, suspend, cancel), audit log, notification generation	All state transitions work; invalid transitions rejected; audit log populated
6	Week 6	Document upload/download API; expiry query endpoint; Swagger UI documentation	File upload/download works; Swagger UI shows all endpoints with auth
7	Week 7	Angular project setup; login page; JWT interceptor; route guards; dashboard (Admin)	Login works end-to-end; JWT stored; protected routes redirect to login
8	Week 8	Policy list, filter, detail, create/edit form pages; state transition buttons	Full policy workflow operable from UI; form validations shown inline
9	Week 9	Customer and Agent management pages; notification bell; Agent and Customer dashboards	All three role dashboards render correctly; notifications appear on state change
10	Week 10	Bug fixes, unit test coverage to 60%+, README documentation, demo preparation	All critical bugs resolved; README with setup instructions; demo recorded

8.1 Definition of Done

- Feature implemented and manually tested via Postman (backend) and browser (frontend)
- Unit tests written for service layer methods
- Code pushed to feature branch, PR raised, and reviewed by mentor
- No critical SonarLint / ESLint errors outstanding
- Swagger docs updated for any new endpoint

KYC	Know Your Customer — identity verification process for compliance
State Machine	A model defining the allowed states and transitions for a policy lifecycle
JWT	JSON Web Token — a compact, signed token used for stateless authentication
RBAC	Role-Based Access Control — permissions assigned based on a user's role

— End of Document —